

Lab 04 – Modbus and OPC UA – Solutions

Industrial Security (Bachelor) • Maps to Section 04 • 45–60 min

LAB 04

🎯 Goal

Lecturer / TA reference: expected PCAP decodes, register / coil values, the comparison-table model answer, and grading rubric.

Part A – Modbus

✓ A1 – Honest read (expected output)

With a freshly started OpenPLC the holding registers are zero-initialised:

```
1 Registers 0..9: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Any non-zero value indicates either a previous lab run that left state behind or a PLC program that writes to %MW0..%MW9. A quick `docker compose restart plc` resets the registers.

✓ A2 – tshark decode of the captured PCAP

`tshark -r lab04-mb.pcap -V -Y modbus` produces one request / response pair per polled block. The Modbus PDU for the read should look like this (abridged):

```
1 Modbus/TCP
2   Transaction Identifier: 1
3   Protocol Identifier: 0
4   Length: 6
5   Unit Identifier: 1
6 Modbus
7   .000 0011 = Function Code: Read Holding Registers (3)
8   Reference Number: 0
9   Word Count: 10
```

The matching response carries the byte count and the ten 16-bit register values:

```
1 Modbus/TCP
2   Transaction Identifier: 1
3   Length: 23
4   Unit Identifier: 1
5 Modbus
6   .000 0011 = Function Code: Read Holding Registers (3)
7   Byte Count: 20
8   Register 0 (UINT16): 0
9   Register 1 (UINT16): 0
10  ... (Register 2..9 also 0)
```

Key teaching point: the entire exchange is cleartext, there is no session, no authentication token, no MAC.

✓ A3 – Inject without authentication

`modbus_inject.py` issues a `Write Single Coil (FC 0x05)` at address 0 with value `0xFF00`. The server acknowledges with an echo of the request – no challenge, no error. Observable side effects:

- In the OpenPLC *Monitoring* page, `QX0.0` flips to `TRUE` (green dot) immediately and stays on.
- A follow-up `modbus_read.py` using `Read Coils (FC 0x01)` returns 1 at address 0.
- Nothing is logged on the PLC – there is no audit trail of the unauthorised write.

This is the core “aha” moment: an unauthenticated, cleartext byte flipped a physical output bit.

Part B – OPC UA

✓ B1 – Sample output of opcua_browse.py

The open62541 example server exposes the standard root and a couple of demo nodes:

```
1 Connected to: opc.tcp://172.28.0.11:4840
2 Browsing Objects:
3   - 0:Server
4   - 1:the.answer
5   - 1:current-time
6   - 1:Demo
```

Exact node names vary slightly between open62541 versions; `0:Server` and at least one user-namespace node (`ns=1`) must be present.

✓ B2 – tshark decode of the OPC UA handshake

`tshark -r lab04-ua.pcap -V -d tcp.port==4840,opcua -Y opcua` shows the textbook OPC UA bring-up:

1. HEL (Hello) from client – advertises buffer sizes and the endpoint URL.
2. ACK (Acknowledge) from server – agrees on buffer sizes.
3. OPN OpenSecureChannel request / response – crucially, `SecurityPolicyURI` is `http://opcfoundation.org/UA/SecurityPolicy#None`.
4. MSG CreateSession + ActivateSession with `UserIdentityToken = Anonymous`.
5. MSG Browse request / response, then CLO.

Because the policy is *None*, every field after the OPN is visible in plaintext in Wireshark – node IDs, browse names, values. There is no signature, no encryption, and no certificate exchange.

1-page comparison – model answer

Dimension	Modbus TCP	OPC UA (this server, defaults)
Authentication	None	Anonymous endpoint enabled
Integrity	None (no MAC)	Sign mode disabled by default
Confidentiality	Cleartext	Cleartext (<code>SecurityPolicy = None</code>)
Operational	Trivial, ubiquitous	Heavier, but capable of TLS + cert auth when configured

Conclusion. OPC UA is *capable* of being secure when configured (e.g. `Aes256_Sha256_RsaPss`, `SignAndEncrypt`, certificate-based user authentication, no anonymous endpoint). The lab server, however, demonstrates the *default* posture, which is no better than Modbus – the wire is still readable and writable to anyone with network reach.

✓ Common student mistakes

- Forgetting the `-d tcp.port==4840,opcua` hint to tshark / Wireshark. Without it the OPC UA frames show up as generic TCP and the dissector never runs.
- Believing TLS is “on by default” in OPC UA. It is not – the spec defines security policies, but the chosen `SecurityPolicy` is whatever the endpoint advertises, and most example servers ship with `None`.
- Confusing FC 0x05 (Write Single Coil) with FC 0x06 (Write Single Register) when reading the injection capture.
- Reading the response byte count (20) as the register count instead of the byte count (each register is 2 bytes).

 Grading rubric

Full marks (100 %) require:

- Both PCAPs handed in (lab04-mb.pcap and lab04-ua.pcap), each containing the relevant request / response pair.
- Annotated hex dump (or tshark -V excerpt) of one Modbus read showing the function code and reference number.
- Comparison table with all four rows (authentication / integrity / confidentiality / operational) filled correctly for both protocols.
- Explicit statement that OPC UA's security is *configuration-dependent* and that the lab server runs with `SecurityPolicy = None` and the anonymous endpoint.

Partial credit: −20 % per missing PCAP, −10 % per missing or wrong table row, −10 % for omitting the configuration-dependent caveat.